

Aufgabe 2: GießRoboter

Teilnahme-ID: 78920

Bearbeiter/-innen dieser Aufgabe:

Tao Zheng

4. September 2026

Inhaltsverzeichnis

1 Klärung der Aufgabenstellung	1
2 Lösungsidee	2
2.1 Generelle Definition des Problems als Graph	2
2.2 Begründung für die Anwendung der ALNS	2
2.3 Aufteilung durch Buckets	3
2.4 Erstmalige Berechnung einer mögliche Lösung	3
2.5 Einsatz der ALNS-Algorithmus	4
2.6 Definition der Zielfunktion	4
2.7 Definition der Zerstörungsfunktion	5
2.8 Definition der Reparaturfunktion	5
2.9 Abschluss	6
3 Umsetzung	6
3.1 Initialisierung	6
3.2 Erstmalige Berechnung einer mögliche Lösung	7
3.3 Erstellung der ALNS Klasse	8
3.4 Zerstörungsoperator	8
3.5 Reparaturoperator	8
3.6 Konfiguration der ALNS-Algorithmus	9
3.7 Abschluss	10
4 Analyse	10
4.1 Laufzeitkomplexität	10
4.2 Speicherplatzkomplexität	13
4.3 Fragestellung Komplexität	13
5 Werkzeuge	15
6 Beispiele	16
6.1 Vorgegebene Beispiele	16
6.2 Eigene Beispiele	20
7 Quellcode	23

1 Klärung der Aufgabenstellung

- „Jeder Roboter fährt von seiner Ladestation aus genau eine Route und kehrt am Ende zu seiner Ladestation zurück.“¹ -> Anhand dieser Aussage wird davon ausgegangen, dass die Roboter sich nicht aufladen können, wenn die Roboter zufällig die Koordinaten der Ladestation betreten. Eine Aufladung bei der Ladestation kann aus logischer Sicht als die Erstellung einer neuen Route anerkannt werden, wodurch die „genau eine Route“ Regelung nicht eingehalten werden kann. Auf Basis der Aussage werden folgende Dinge als Beschluss vorgenommen:

¹Quelle: <https://bwinf.de/bundeswettbewerb/44/faq-2-runde-1/>, Geöffnet am 09.04.2026

1. Die Angabe der Position der Ladestation muss auf der Strecke zwischen dem ersten Baum sowie dem letzten Baum einer Route sein, weshalb für die Koordinaten der Ladestation die Koordinaten des ersten Baums einer Route ausgegeben werden.
2. Beim zufälligen Betreten der Ladestation-Koordinaten (unter zufällig wird gemeint, dass der Roboter die Ladestation nicht nach dem Abschluss der Route betritt) behandelt der Roboter die Ladestation als nicht-existierend und fährt sinngemäß vorbei.

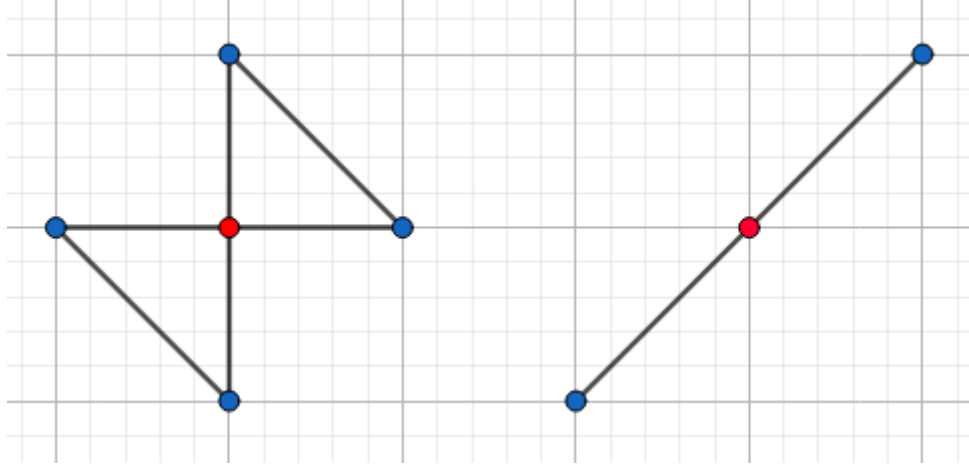


Abbildung 1: Die blauen Punkte stellen die Bäume dar, die roten Punkte stellen die Ladestationen dar, und die schwarzen Linien stellen eine mögliche Route dar.

3. Eine Route kann keine Strecke sein, die beim letzten Baum endet. Stattdessen muss eine Route immer ein Kreis sein, der nach Abschluss des letzten Baums einer Route zurück zur Ladestation bewegt. Da aber aus Punkt 1. meine Ladestation immer beim ersten Baum ist, bedeutet es, dass der Roboter nach Abschluss des letzten Baums zum ersten Baum zurückkehrt.

2 Lösungsidee

Um das Problem zu lösen, wird ein metaheuristischer Ansatz kombiniert mit der „Adaptive Large Neighborhood Search“ (ALNS) sowie „Simulated Annealing“ verwendet, um eine mögliche gute Lösung für das Problem zu generieren.

2.1 Generelle Definition des Problems als Graph

Durch das Problem ist der vollständige Graph $G = (V, E)$ gegeben, mit den Knoten (bzw. Bäumen) $V = \{v_1, v_2, \dots, v_n\}$ sowie Kanten E . Die Anzahl der Knoten im Graph G wird mithilfe von $n \in \mathbb{N}$ definiert. Jede Kante $E_{i,j}$ erhält den Distanz- bzw. Kostenfunktion $w_E : E \rightarrow [0; \infty[$.

Zusätzlich gibt es noch die Mengen der Routen $R \subseteq V$, die eine Menge an Knoten besucht, wobei jeder Knoten in einer Route beinhaltet werden muss. Alle Routen erhalten zusätzlich noch die Kostenfunktion $w_R : R \rightarrow [0; \infty[$, die die Länge der Route R berechnet. Für die Länge einer Route gilt die Begrenzung: $w_{R(r)} \leq C \forall r \in R$, wobei C die maximale Länge einer Route angibt. Ziel ist dabei, die Anzahl der Routen $|R|$ zu minimieren.

2.2 Begründung für die Anwendung der ALNS

Für die Begründung der Verwendung des ALNS, wird auf das Kapitel 4.3 „Fragestellung Komplexität“ verwiesen. Es wird empfohlen, sich dieses Kapitel durchzulesen, bevor die Lösungsidee weitergelesen wird, da einige Punkte auch auf die Begründung referenziert werden.

2.3 Aufteilung durch Buckets

Um die minimale Anzahl an Routen zu berechnen, hilft es mehrere optimale Routen zu generieren, da dadurch für jede Route mehr Knoten im Durchschnitt aufgenommen werden können, als in Vergleich zur ungünstigen Route. Um mögliche, optimale Routen ausrechnen zu können, muss der Algorithmus die Distanz zwischen zwei verschiedenen Knoten wissen, um den nahen Knoten aufzunehmen. Aufgrund der sehr hohen Anzahl an Knoten würde eine komplette Distanzberechnung im vollständigen Graphen G eine Laufzeitkomplexität i. H. v. $\mathcal{O}(N^2)$ verursachen, was zu vermeiden ist. Aus diesem Grund wird eine Einschränkung für die Knoten mithilfe von Buckets eingebaut, die alle Knoten dazu zwingen, Kanten nur zwischen einem anderen nahe gelegenen Knoten zu bilden. Durch diese Einschränkung können die kürzesten Verbindungen zwischen zwei Knoten in einer deutlich geringeren Laufzeit berechnet werden.

Am Anfang der Berechnung wird eine zwei-dimensionale Bucket B erstellt, die jedem einzelnen Knoten v_i basierend auf ihre x- und y-Koordinaten einen Speicherort im Bucket ($b : V \rightarrow B; B_{j,k} \in B$, wobei j und k für den Koordinatenbereich darstellen) zuordnet. Dabei stellt ein Speicherort eine Fläche (z.B.: $40 \leq x \leq 80 \wedge 120 \leq y \leq 160$) dar, die mehrere Knoten aufnehmen kann (N Knoten zu 1 Bucket Beziehung). Anschließend wird eine Liste $L_{v_i} \subseteq V$ generiert, die für alle Knoten eine Menge an benachbarten Knoten angibt, wobei die Berechnung der Menge L_{v_i} mithilfe der zuvor generierten Buckets erfolgt. Dabei wird für den Knoten v_i folgende Operation durchgeführt, um benachbarte Knoten zu erfassen:

$$L_{v_i} = \sum_{j=-d}^d \sum_{k=-d}^d B_{j+j_b(v_i), k+k_b(v_i)} \quad (1)$$

Der Wert d entspricht der Reichweite der zu überprüfenden Buckets. Diese wird so lange erhöht, bis $|L_{v_i}| \geq \min(10, |V|)$ gilt. D.h., es werden mind. 10 oder alle Knoten als Nachbarn eingespeichert.

Hierbei gibt es noch den kurzen Hinweis, dass durch diese Abfrage entstehende Nachbarn nicht immer die absolut nächsten Nachbarn sein müssen. Das liegt daran, dass diese Abfrage in Form eines Quadrats erfolgt, wodurch auch Knoten, die an einer Ecke des Quadrats befinden, mit aufgenommen werden. Um den wirklich absolut nächsten Nachbarn zu erfassen, müsste daher normalerweise ein Kreis gebildet werden. Jedoch kann man dies in diesem Fall des Problems vernachlässigen, da es einerseits durch die Konfiguration recht selten ist und auch wenn so etwas passieren würde, weiterhin - unabhängig davon - gute Ergebnisse entstehen werden.

2.4 Erstmalige Berechnung einer mögliche Lösung

Um für den späteren ALNS Einsatz vorzubereiten, wird eine mögliche Lösung mithilfe eines Greedy Verfahrens berechnet. Alle Knoten v_i werden aufsteigend nach der Summe der x- sowie y-Koordinaten sortiert und auf die Knoten eine Route R erstellt. Mithilfe der Formel:

$$\min_{v_j \in L_{v_i}, v_j \notin R} (w_E(v_i, v_j)) \quad (2)$$

können mögliche Knoten v_j gesucht werden, die dann zu einer optimalen Route gebildet werden. Dabei wird geachtet, dass durch das Hinzufügen der Knoten die Anforderung:

$$w_E(v_0, v_{|R|-1}) + \sum_{i=0}^{|R|-1} (w_E(v_i, v_{i+1})) \leq C; v_i \in R \quad (3)$$

erfüllt, um die maximal erlaubte Länge nicht zu überschreiten. (Dabei sind die einzelnen Knoten v_i in der Besuchsreihenfolge der Route R eingespeichert worden. v_0 steht für den Startknoten und $v_{|R|-1}$ für den letzten Knoten der Route, bevor es wieder zur Ladestation am Knoten v_0 zurückkehrt.)

Die Generierung einer möglichen Lösung wird die Berechnungsdauer des ALNS-Algorithmus stark verringern.

2.5 Einsatz der ALNS-Algorithmus

Bei dem ALNS-Algorithmus werden mehrere Zustände für die Routen erstellt, die sich bei der Verteilung der Knoten unter den Routen unterscheiden können. Mithilfe einer Zielfunktion wird die beste generierte Route gespeichert. Der ALNS-Algorithmus arbeitet mit zwei Operatoren:

- Zerstörungsoperatoren: Entfernt zufällige Knoten aus deren Route und markiert die Knoten als „nicht zugewiesen“
- Reparaturoperatoren: Nimmt alle Knoten aus der „nicht zugewiesen[e]“ Menge und weist ihnen eine nahe Route zu oder erstellt eine neue Route für diese Knoten.

Diese Operatoren werden mithilfe der „RouletteWheel“ Verfahren ausgewählt.

Die Überprüfung, ob ein Zustand akzeptiert oder verworfen wird, erfolgt mithilfe des „SimulatedAnnealing“ Algorithmus.² Das liegt daran, dass bei großen Problemen mehrere lokale Minimums unter den Lösungen geben könnte, die davon abhalten, bessere Lösungen zu finden, weshalb man auch schlechtere Lösungen annehmen sollte, um aus solchen lokalen Minimums zu entkommen. Jedoch muss man hier wieder bedenken, dass dies wiederum keine globalen Minimums garantieren muss, da bei sehr großen Problemen sehr viele lokale Minimums geben könnte.³

Als Abbruchbedingung kann eine maximale Berechnungszeit verwendet werden.

2.6 Definition der Zielfunktion

Um das Problem zu lösen, sollen auf folgende Sachen eine Strafe in Abhängigkeit der maximalen Routenlänge C zugewiesen werden:

- Für jede nicht zugeordnete Baum ($1.5x$ von C) (Hauptziel)
- Für jede hinzugefügte Route ($1.3x$ von C) (Hauptziel)
- Für jede Länge der Routen ($0.0x - 1.0x$ von C) (Nebenziel)

Daraus folgt die folgende Formel, die minimiert werden soll:

$$\min \left(N_{\text{NzB}} \cdot 1.5C + N_R \cdot 1.3C + \sum_{k=0}^{N_R-1} \left(w_E(v_0, v_{|R_k|-1}) + \sum_{i=0}^{|R_k|-1} (w_E(v_i, v_{i+1})) \forall v_i \in R_k \right) \right) \quad (4)$$

Dabei stehen die Werte: N_{NzB} für die Anzahl der nicht zugewiesenen Knoten, N_R für die Anzahl der Routen.

Die Bestrafung i.d.H.v. $1.5 \cdot C$ für jeden nicht zugeordneten Baum fördert die Reparatur des Graphen, da es immer günstiger ist, einem nicht zugeordneten Baum eine Route zuzuweisen. Das liegt daran, dass beim Erstellen einer Route die Strafe genau $1.3 \cdot C$ beträgt und die Strafe für die fehlende Zuweisung i.d.H.v. $1.5 \cdot C$ aufgehoben wird (da die Länge der Route 0 ist, werden keine Strafen für die Routenlänge verhängt) und beim Einfügen zu einer existierenden Route die Strafe nur höchstens C betragen kann und hier wieder die Strafe für die fehlende Zuweisung ($1.5 \cdot C$) aufgehoben wird.

Bei beiden resultiert daher $1.3C < 1.5C$ und $C < 1.5C$. Hinzu kommt noch, dass eine mögliche vollständige Lösung an das ALNS abgegeben wird, die ebenfalls in der Bewertung beachtet wird. Würde man eine vollständige Lösung zerstören, so wäre die Strafe (anhand der Umkehrung der oben genannten Gleichung) höher als die Strafe der vollständigen Lösung.

Betrachtet man die Strafe für die Anzahl der Routen, so ist es ebenfalls günstiger, bei Möglichkeit Routen aus einzelnen Knoten aufzulösen und diese Knoten in andere bereits existierende Routen hinzuzufügen. Die Strafe einer Route beträgt $1.3C$, während bei einer Einfügung eine Strafe von höchstens C beträgt ($\rightarrow C < 1.3C$). Werden Routen aus mehreren Knoten aufgelöst, so hängt es natürlich davon ab, ob die neuen Routen alle Knoten aufnehmen können und gleichzeitig noch weiterhin optimal bleiben.

²Vgl. https://de.wikipedia.org/wiki/Simulierte_Abk%C3%BChlung

³Vgl. https://en.wikipedia.org/wiki/Very_large-scale_neighborhood_search

Natürlich kann es sein, dass nach dem Entfernen von vielen Knoten aus vielen Routen eine Lösung entsteht, die weniger Routen verwendet, aber trotzdem aufgrund der Distanzen eine höhere Gewichtung verwendet. Jedoch habe ich mich trotzdem für das Modell so entschieden, da diese Einstellung einen positiven Einfluss auf das Simulated Annealing Verfahren hat. Das liegt daran, dass die Strafen für die Zerstörung der Knoten sowie das Ausprobieren von neuen Routen nicht so hoch sein sollten, da die Verfahren sich sonst vor Möglichkeiten abschrecken könnten, die womöglich zu einem besseren lokalen Minimum führen würden. Hinzu kommt, dass diese Einstellung beim Ausprobieren das beste Ergebnis in einer kurzen Zeit geliefert haben.

2.7 Definition der Zerstörungsfunktion

Bei der Zerstörungsfunktion wird ein neuer Zustand erzeugt, wobei in dem neuen Zustand zufällige Knoten aus zufälligen Routen entfernt werden und die entfernten Knoten werden als „nicht zugewiesen“ markiert. Dabei wird eine mathematische Funktion angewendet, die, abhängig von der Anzahl der Bäume, beschreiben soll, wie viel Prozent der Bäume höchstens aus ihren Routen entfernt werden sollen. Diese mathematische Funktion wird wie folgt beschrieben:

$$\begin{cases} 0.2 & \text{für } x \leq 50 \\ 0.2 - 0.0011(x - 50) & \text{für } 50 < x \leq 186 \\ 0.05 & \text{für } 186 < x \end{cases} \quad (5)$$

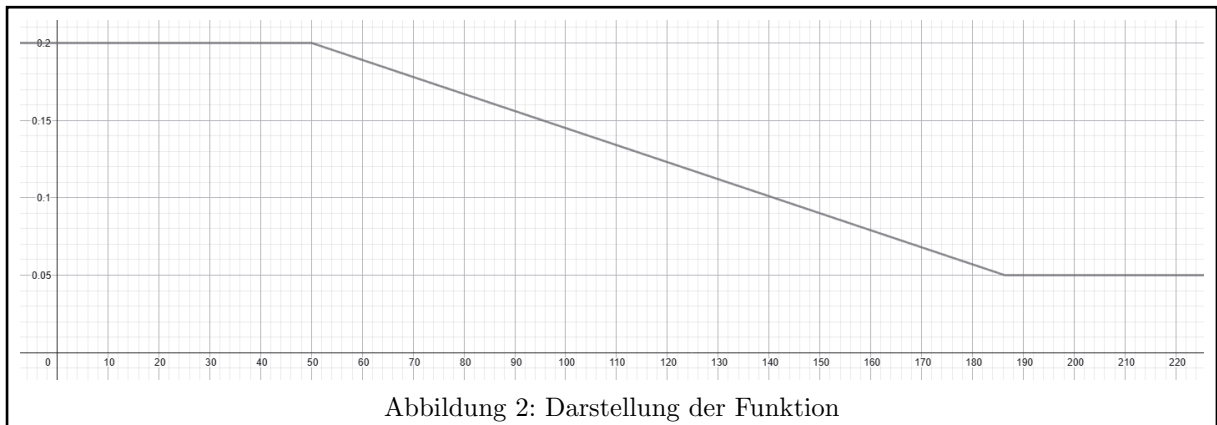


Abbildung 2: Darstellung der Funktion

Bei geringen Höchstmengen der Bäume soll dadurch der Algorithmus motiviert werden, deutlich mehr Bäume zu entfernen und dadurch auch mehr Möglichkeiten auszuprobieren, da die absolute Laufzeitkomplexität bei geringer Anzahl an Bäumen sehr gering ist. Bei höherer Anzahl an Bäumen soll der standardmäßige Zerstörungsgrad angewendet werden.

Durch die Multiplikation der Funktion mit der Anzahl der Bäume N kann die Höchstzahl $N_{d_{\max}}$ der zu zerstörenden Bäume berechnet werden. In diesem Verfahren wird daher eine zufällige Zahl zwischen 1 und $N_{d_{\max}}$ ausgewählt und anschließend so viele Bäume aus ihren Routen entfernt.

Die durch die Entfernung eines Knotens aus einer Route entstandene Route erfüllt immer die Bedingung, dass die Länge der Route sich weiterhin innerhalb C befindet. Das liegt daran, dass durch den 2-dimensionalen Koordinatenraum alle Knoten einer Route nie mehr als $0.5C$ zueinander entfernt sind und bei Anwendung des Kosinussatzes mit der Vorgängerstrecke, Nachfolgerstrecke sowie den Winkeln zwischen den beiden Strecken der zu entfernenden Knoten immer kleiner oder gleich der Summe der beiden Strecken sind.

$$\sqrt{a^2 + b^2 - 2ab \cdot \cos(\gamma)} \leq a + b \quad (6)$$

Somit ist es bewiesen, dass die Zerstörungsfunktion zu keinen ungültigen Routen führen kann.

2.8 Definition der Reparaturfunktion

Während des Reparaturvorgangs werden den „nicht zugewiesen[en]“ Bäumen eine Route zugewiesen. Dabei wird die Reihenfolge der Einfügung zufällig bestimmt. Die Bäume v_i werden an eine Route

zugewiesen, die von einem benachbarten Baum (aus L_{v_i}) aktiv verwendet wird, wobei hierbei wieder beachtet wird, dass beim Einfügen die maximale Länge wiederum nicht überschritten wird. Welche Route hierbei verwendet wird, wird ebenfalls per Zufall entschieden. Konnte der Baum keiner Route zugewiesen werden (weil alle Routen ihre maximale Länge erreicht haben oder weil keine Routen in der Nähe gibt), so erstellt der Baum v_i eine neue Route.

2.9 Abschluss

Dieses Verfahren erlaubt schlussendlich neue Möglichkeiten an Routen sowie Verbindungen zu probieren. Mithilfe der Zielfunktion werden die besten Ergebnisse gespeichert, die eine geringe Anzahl an Routen verwenden, wodurch auch die Anzahl der benötigten Roboter verringert werden kann, um alle Bäume aus dem Gießroboter Problem zu versorgen. Sobald die maximale Berechnungsdauer erreicht wurde, werden alle Ergebnisse im vorgegebenen Format ausgegeben.

3 Umsetzung

Für die Umsetzung wurde das Programm in Python geschrieben. Für die Berechnung der Distanzen zwischen zwei Bäumen wird die Funktion `calc_dist(a, b)` mithilfe des Satz des Pythagoras definiert. Zum Thema Umgang mit Ungenauigkeiten mit Kommazahlen habe ich mich zu folgenden Dingen entschlossen:

- Es werden ausschließlich nur die Distanzen zwischen zwei verschiedenen Knoten dauerhaft gespeichert.
- Um viele langfristige Additionen sowie Subtraktionen einer Route zu vermeiden, werden die Längen einer Route nur kurzfristig gespeichert. Bei Änderungen einer Eigenschaft aus einer Route wird die Länge der Route komplett neu bestimmt. Dabei werden keine Zwischenergebnisse aus vorherigen Berechnungen der Route übernommen.

3.1 Initialisierung

Während die Daten eingelesen werden, werden folgende Variablen verwendet:

- `maxDist`: Entspricht die maximale erlaubte Länge einer Route C .
- `nTrees`: Entspricht die Anzahl der Bäume $|V|$.
- `positions`: Speichert in einer Liste alle x- und y-Koordinaten eines Baumes ein.
- `minx, miny`: Die kleinsten x- und y-Werte der Bäume.
- `maxx, maxy`: Die größten x- und y-Werte der Bäume.

Die Werte für `minx`, `miny`, `maxx`, `maxy` sind für die Bestimmung der Dichte da und werden für die Erstellung der 2 dimensionale Buckets verwendet. Abhängig von der Dichte wird ein Wert für `splitfactor` ausgewählt, welcher die Länge eines Buckets beschreibt. Dabei wird folgende Formel verwendet:

$$\text{density} = \frac{|V|}{(\text{maxx} - \text{minx}) \cdot (\text{maxy} - \text{miny})} \cdot 100$$

$$\text{splitfactor} = \begin{cases} 100 & \text{für density} \leq 0.1 \\ 49 & \text{für density} \leq 0.5 \\ 23 & \text{für density} > 0.5 \end{cases} \quad (7)$$

Anschließend wird mithilfe von Numpy ein 2-dimensionales Buckets-System erstellt, die in allen Buckets eine Liste beinhalten. Die dafür benötigten Dimensionen entsprechen folgender Formel:

$$\text{für } x : \left\lceil \frac{\text{maxx} - \text{minx}}{\text{splitfactor}} \right\rceil + 1 \quad (8)$$

$$\text{für } y : \left\lceil \frac{\text{maxy} - \text{miny}}{\text{splitfactor}} \right\rceil + 1 \quad (9)$$

Dadurch können alle benötigten Buckets erstellt werden. Mithilfe von `numpy.frompyfunc(list, 0, 1)` können in alle der Buckets eine Liste als Datentyp verwendet werden, um später die einzelnen Knoten in diese Listen einzuspeichern. Nach der Erstellung der Buckets können die einzelnen Bäume abhängig von deren Koordinaten in ihren Buckets zugewiesen werden. Die Zuweisungskordinaten entsprechen:

$$\text{für } x : \left\lfloor \frac{x_{v_i} - \min x}{\text{splitfactor}} \right\rfloor \quad (10)$$

$$\text{für } y : \left\lfloor \frac{y_{v_i} - \min y}{\text{splitfactor}} \right\rfloor \quad (11)$$

Mithilfe der Buckets kann anschließend die Hilfsfunktion `getNearbyTrees(id, d)` definiert werden, die alle in der Nähe liegenden Bäume (die sich in den Buckets nicht weiter als d -Buckets von dem gefragten Baum entfernt sind) zurückgibt. Dabei wird eine Eigenschaft aus der Numpy Funktion verwendet, die einen 2-dimensionalen Bereich zurückgeben kann. Anhand dieser Bereiche werden die einzelnen Bäume entnommen und so zurückgegeben. Diese Hilfsfunktion wird für alle Bäume angewendet, wobei alle benachbarten Bäume aus einem Baum v_i in der Liste `nearbyList` ($\rightarrow L_{v_i}$) gespeichert werden, um die Laufzeit zu verringern. Dabei wird darauf geachtet, dass entweder mind. 10 oder $|V|$ Bäume als benachbart gekennzeichnet werden.

Anschließend wird noch die Funktion `dist(a, b)` definiert, die die Länge zwischen Baum a und b zurückgibt. Dabei werden alle Ergebnisse in `storage` gespeichert, um schnellere Berechnungszeiten bei bereits berechneten Längen zu ermöglichen. Storage besteht aus einer Liste mit Hashmaps, da durch die Buckets nicht alle Knoten zueinander verbunden sind. Dies vermeidet in den meisten Fällen eine $\mathcal{O}(N^2)$ Speicherplatzkomplexität. Alle Distanzen werden mit der Hilfsfunktion `calc_dist(a, b)` berechnet, die die Formel des Satz des Pythagoras verwendet.

3.2 Erstmalige Berechnung einer mögliche Lösung

Bei der erstmaligen Berechnung sind alle darauf bezogenen Variablen mit den Wörtern `pre_` sowie `init_` markiert.

Um eine Lösung zu berechnen, werden zunächst einmal alle Knoten basierend auf die Summe ihrer beiden Koordinaten sortiert. Das liegt daran, dass die Durchführung eines Greedy Algorithmus aus einem zufälligen Knoten im Graphen zu schlechteren Ergebnissen führen könnte als die Durchführung des Algorithmus in einer Reihenfolge, die durch ihre Position bestimmt wird. Um einen benutzerdefinierten Sortier-Algorithmus in Python zu erstellen, wird von `functools` die `cmp_to_key` Funktion importiert. Anschließend werden alle Knoten sortiert und unter `pre_positions` gespeichert. Die Variable `pre_calculated` überprüft, ob einem Knoten schon bereits an eine Route zugewiesen wurde. In `init_Routes` werden alle erstellten Routen gespeichert und alle Knoten haben einen Verweis auf ihre Route per `init_assignment`.

`init_assignment` soll es möglich machen, aus den benachbarten Knoten `nearbyList` schnell deren Route zu bekommen. Dies wird später die Reparaturoperatoren im ALNS zu vereinfachen, da man sonst durch alle einzelnen Knoten aller Routen durchgehen müsste, um aus einem Knoten dessen Route zu bestimmen.

Während der Greedy Verfahren werden die noch nicht zugewiesenen Knoten in der Reihenfolge von `pre_positions` bearbeitet. Während des Verfahrens wird der in die Route zuletzt eingefügte Knoten verwendet, um mithilfe von `nearbyList` den nächstgelegenen Nachbarn zu bestimmen. Nachdem ein guter Knoten gefunden wurde, wird überprüft, ob das Hinzufügen der Knoten den Routelimit `maxDist` nicht überschreitet. Solange es weiterhin innerhalb des Limits ist, werden neue Nachbarn immer hinzugefügt. Falls es der Limit nicht eingehalten wird, wird die Route als abgeschlossen gesehen und ebenfalls bei `init_Routes` abgegeben und die Verweise der Knoten auf ihre Route (`init_assignment`) werden erstellt.

Hier gibt es den Hinweis von mir, dass beim Abschließen einer Route nicht immer heißen muss, dass es keine weiteren Knoten zum Einfügen gibt. Das liegt daran, dass die Route ein Kreis ist und deshalb auch von der Distanz der beste Knoten zu den ersten Knoten (Ladestation) abhängt. Bei der Suche nach den guten Knoten wurde diese Distanz absichtlich vernachlässigt, weshalb es weiterhin freie Knoten geben könnte, die zwischen dem zuletzt eingefügten Knoten und den ersten Knoten einer Route liegen.

3.3 Erstellung der ALNS Klasse

Um den ALNS Algorithmus anzuwenden, wird die Klasse `VRP` (CVRP) erstellt, die einen möglichen Zustand beschreibt. Für die Klasse selbst wurde folgende Vorlage verwendet: https://alns.readthedocs.io/en/latest/examples/capacitated_vehicle_routing_problem.html#Solution-state

- Beim Initialisieren eines Zustands werden die Routen `routes`, die Liste der nicht zugewiesenen Knoten `unassigned` sowie die einzelnen Zuweisungen zu den Knoten `assignment` gespeichert.⁴
- In der Liste `unassigned` werden alle entfernten Knoten gelagert. Für jeden Knoten in `unassigned` wird eine Strafe in der Höhe von $1.5C$ verteilt (vgl. Lösungsidee - Definition der Zielfunktion). Diese Liste wird später für den Reparaturopoperator verwendet, um alle entfernten Knoten wieder einzufügen, mit der Hoffnung, auf bessere Kombinationen zu kommen.
- Die Funktion `objective` beschreibt die in der Lösungsidee genannte Zielfunktion, welche für die Bewertung der Bestrafungen verwendet wird.⁵
- `route_cost` berechnet aus einer Route die Länge der Route aus, indem die Länge der Route unter der Verwendung der einzelnen Knoten iterativ addiert werden.⁶

3.4 Zerstörungsoperator

Während der Definition wurden einige Konstanten erstellt, die für den Zerstörungsoperator verwendet werden:

- `degree_of_destruction`: Beschreibt den maximalen prozentualen Anteil der zu zerstörenden Bäume. (Vgl. Lösungsidee 2.7)⁷
- `trees_to_remove` beschreibt die tatsächliche, maximale Anzahl der Bäume, die entfernt werden sollen.⁸

Dadurch kann die `multi_random_removal` Funktion erstellt werden, die am Anfang eine Kopie unter der Verwendung der ursprünglichen Instanz erstellt (`destroyed`). Anschließend wird eine Zahl zwischen 1 und dem Wert von `trees_to_remove` bestimmt, die die Anzahl der Bäume zerstören soll. Danach wird mithilfe von `randint` eine zufällige Route aus `destroyed.routes` ausgewählt, und aus dieser Route wird wieder ein zufälliger Knoten ausgewählt, der aus der Route entfernt wird. Die Entfernung wird durchgeführt, indem man die Knoten in `destroyed.unassigned` hinzufügt, die Knoten aus der Route entfernt und anschließend die Referenzen von den Knoten zu den Routen im `destroyed.assignment` entfernt.⁹

Durch diesen Einsatz werden Routen mit mehreren Knoten belohnt, während Routen mit wenigen Knoten bestraft werden. Das liegt daran, dass alle Routen unabhängig von der Anzahl der Knoten in der Route die gleiche Wahrscheinlichkeit haben, von der Zerstörung betroffen zu sein. Routen mit mehreren Knoten können dadurch mehrere Löschungen überstehen, während Knoten mit wenigen Knoten ganz verschwinden können (wenn der letzte Knoten aus der Route gelöscht wird). Dadurch wird die Bildung einer Route aus mehreren Knoten gefördert, und Routen aus wenigen Knoten können womöglich ganz aufgegeben werden und sich in andere Routen integrieren.

Sobald die Löschung abgeschlossen ist, wird der neue Zustand zurückgegeben.

3.5 Reparaturopoperator

Mithilfe der Funktion `fast_repair` werden alle nicht eingefügten Knoten (alle Knoten aus `unassigned`) in eine zufällige Reihenfolge (mithilfe von `shuffle(unassigned)`) in die Routen hinzugefügt. Für jeden nicht hinzugefügten Knoten v_i werden durch alle benachbarten Knoten (mithilfe von `nearbyList[v_i]`) durchgegangen. Die Reihenfolge der benachbarten Knoten wird ebenfalls zufällig neu bestimmt, wodurch mehr Möglichkeiten für einen Knoten entstehen. In der Route der benachbarten Knoten wird versucht,

⁴Funktion wurde aus der Vorlage angepasst.

⁵Funktion wurde aus der Vorlage angepasst.

⁶Name wurde aus der Vorlage übernommen, jedoch wurde die Funktion von mir komplett verändert.

⁷Name wurde aus https://alns.readthedocs.io/en/latest/examples/capacitated_vehicle_routing_problem.html#Destroy-operators übernommen, wobei ich hier den Wert komplett überarbeitet habe.

⁸Name sowie Wert wurden aus https://alns.readthedocs.io/en/latest/examples/capacitated_vehicle_routing_problem.html#Destroy-operators komplett übernommen.

⁹Als Vorlage wurde der Code aus https://alns.readthedocs.io/en/latest/examples/capacitated_vehicle_routing_problem.html#Destroy-operators verwendet, wobei die meisten Sachen von mir komplett verändert wurden.

die Knoten v_i an der ersten möglichen Position unter der Beachtung der Maximalen hinzuzufügen. Sobald eine mögliche Position gefunden wird, wird diese Knoten - unabhängig davon, ob es günstig ist oder nicht - hinzugefügt (d.h. die Referenzen zu der Route wird für die Knoten v_i erstellt -> **assignment** und in der Route hinzugefügt). Falls an keine Stelle einer Route die Knoten v_i hinzugefügt werden kann, so wird eine andere Route verwendet, die in der Verbindung mit einem anderen benachbarten Knoten steht. Falls an keine benachbarte Route ein Knoten hinzugefügt werden kann, so entsteht an diesem Knoten eine neue Route.

Für den Fall, dass keine benachbarte Route die Knoten aufnehmen kann, muss man dabei bedenken, dass aufgrund der Limitierung der Routen zu den benachbarten Knoten womöglich weitere Routen existieren könnten, die die Knoten aufnehmen könnten. Jedoch ist es schwer vorzustellen, solche Route als optimal zu bezeichnen, wenn diese Route nicht in Verbindung mit den nahen Knoten steht. Hinzu kommt noch, dass durch das Buckets System öfters mehr als nur 10 nahe Knoten (wie z.B.: 40) aufgelistet werden, weshalb in den aller meisten Fällen eine mögliche optimale Route durch die benachbarten Knoten verlaufen sollte. Aus diesem Grund können weiter entfernte Routen vernachlässigt werden, auch wenn diese in der Lage sein könnten, die Knoten aufzunehmen.

Nach Abschluss aller Einfügungen wird der Zustand zurückgegeben.

3.6 Konfiguration der ALNS-Algorithmus

Für den ALNS wird eine **RandomState** als zufallsgenerator verwendet, welches den Seed 42 verwendet.

Der Seed wurde ursprünglich beim Testen aus https://alns.readthedocs.io/en/latest/examples/alns_features.html übernommen. Da ich diese Aufgabenstellung dauerhaft mit diesem Seed gearbeitet habe, hatte ich den Seed nie abgeändert. Natürlich ist es möglich, einen Seed so auszuwählen, dass das Endergebnis besser wird (als Beispiel kann das IOI 2025 Problem: Triple Peaks Problem Parts 2 verwendet werden (vgl. <https://ioinformatics.org/files/ioi2025problem2.pdf>), das als eine mögliche Lösung die Verwendung eines Zufallsgenerators, kombiniert mit einem guten Seed, gelöst werden kann. Der Quellcode der Lösung des IOI Problems ist in der „Solutions to all problems“ (<https://ioinformatics.org/files/ioi2025solutions.zip>) zu finden). Jedoch kann man dies hier vernachlässigen, da:

1. Eine angepasste Seed nur für feste, vorgegebene Probleme effektiv wäre und bei anderen Problemen womöglich kaum Einfluss auf bessere Ergebnisse haben.
2. Diese Lösung soll auch für unbekannte nicht vorgegebene Probleme effektiv ohne Bias funktionieren.

Aus diesem Grund wird daher auf eine angepasste Seed verzichtet und 42 als Seed weiterverwendet.

Als Gewichtung (Belohnung) für die einzelnen Operatoren wird **RouletteWheel** mit folgender Konfiguration verwendet:¹⁰

- Gewichtung für das Finden eines neuen globalen Minimums: 3
- Gewichtung für das Finden einer besseren Lösung: 1
- Gewichtung für das Akzeptieren einer Lösung: 1
- Gewichtung für das Ablehnen einer Lösung: 0.1
- Decay: 0.8

Die Frage, ob ein Zustand angenommen wird, erfolgt über **SimulatedAnnealing**¹¹ mit folgenden Konfigurationen:

- Starttemperatur: 0.8
- Endtemperatur: 0.01
- Schritte: 0.85
- Methode: Exponenziell

¹⁰Die Idee wurde aus https://alns.readthedocs.io/en/latest/examples/alns_features.html übernommen. Die Konfigurationen wurden von mir selbst durch das Testen komplett abgeändert.

¹¹Vgl. https://de.wikipedia.org/wiki/Simulierte_Abk%C3%BChlung

Dadurch soll genug Freiraum geben, andere - schlechtere - Zustände anzunehmen um so auf bessere Lösungen zu kommen. Es sollen die Möglichkeiten entstehen, auch aus lokalen Minimums herauszukommen, um anderen, besseren lokalen Minimums anzunähern. Als Stop-Bedienung wird `MaxRuntime` verwendet. Der ALNS-Algorithmus beendet automatisch nach 0.2 Sekunden.

Beim Testen kam heraus, dass für die allermeisten Beispiele schon bereits nach 0.2 zu gute Lösungen führen. Bei deutlich längerem Warten werden die Ergebnisse natürlich meistens verbessert, jedoch möchte ich die meisten Beispiele innerhalb von einer Sekunde lösen, weshalb ich die Konfiguration i.d.H.v. 0.2 Sekunden als optimal halte.

3.7 Abschluss

Nachdem der ALNS-Algorithmus durchgeführt wurde und ihre Stop-Bedienung i.d.H.v. 0.2 Sekunden erreicht hat, wird der beste Zustand mit der minimalen Strafe ausgegeben. Die Anzahl der Routen entspricht der Anzahl der (nicht leeren) Einträge aus `Zustand.routes`. Für die Länge einer Route wird hier wieder `route_cost` verwendet, und die Koordinate einer Ladestation entspricht der Koordinate des ersten Baums.

4 Analyse

Aufgrund der Verwendung des ALNS als externes Programm lässt sich nun die Laufzeitkomplexität nicht mehr eindeutig bestimmen. Auf der Seite der Dokumentation sowie auf der Github Seite des Entwicklers wurden keine Laufzeit- und Speicherkomplexitäten angegeben. Aus diesem Grund wurde für diesen Bereich Gemini verwendet. Aus Transparenzgründen ist im Abgabeordner das Dokument „Anhang1.pdf“ zu finden, welches den Chatverlauf mit Gemini aufzeigt.

4.1 Laufzeitkomplexität

Im Bereich der Laufzeitkomplexität lässt sich das Problem in mehrere Teile aufteilen:

- Die Erstellung einer 2-dimensionalen Buckets
- Die Zuweisung der einzelnen benachbarten Knoten
- Die Erstellung einer ersten möglichen Lösung
- Die Durchführung des ALNS Algorithmus (Zerstörungs- und Reparaturoperator)

Bei der Erstellung des Buckets werden mehrere Listen abhängig von den kleinsten sowie größten absoluten Beträgen der Koordinaten erstellt. Hinzu kommt noch, dass abhängig von der Dichte der Knoten die Liste mithilfe von „splitfactor“ einen größeren oder kleineren Bereich darstellen. Aus diesem Grund kann in diesem Fall die Laufzeitkomplexität beim Erstellen der Buckets wie gefolgt beschreiben:

$$\mathcal{O}\left(\frac{\Delta X \cdot \Delta Y}{\text{splitfactor}}\right) \quad (12)$$

Dabei gelten: $\Delta X = x_{\max} - x_{\min}$ und $\Delta Y = y_{\max} - y_{\min}$, die alle den Unterschied zwischen den größten und kleinsten Koordinaten der Bäume darstellt. Der Wert des splitfactor ist mit geringer Dichte höher, während es bei hoher Dichte geringer wird (vgl. Lösungsidee).

Für die Suche aller benachbarten Knoten werden die erstellten Buckets verwendet, die wiederum nochmals aufgeteilt werden. Dabei wird der Prozess für jeden Baum durchgegangen. Der Suchradius entspricht dabei dem Wert d , alle Buckets, die höchstens d Buckets von der gefragten Bucket entfernt sind. Abhängig von dem Wert von d werden daher folgende Anzahl an Buckets untersucht:

d	0	1	2	3	4	5	6
Anzahl der Buckets	1	9	25	49	81	121	169

Das liegt daran, dass bei der Bucket Abfrage immer ein Quadrat entsteht, dessen Fläche mit folgender Formel beschrieben werden kann.

$$\text{Anzahl der Buckets} = (1 + 2 \cdot d)^2 \quad (13)$$

In den Buckets drinnen werden außerdem noch die einzelnen Identifikationsnummern der Bäume extrahiert, wobei man hierbei jedes Bucket einzeln besichtigen muss. Durch die Limitierung werden bereits ab 10 Bäumen das Suchprozess beendet. Aus diesem Grund besitzt es in den durchschnittlichen Fällen die Laufzeitkomplexität:

$$\mathcal{O}(N \cdot \bar{d}^2 \cdot \bar{L}) \quad (14)$$

N beschreibt die Anzahl der Bäume, $\bar{d}^2$ beschreibt die durchschnittliche Distanz, die während der Suche verwendet wurde und $\bar{L}$ beschreibt die durchschnittliche Anzahl der Nachbarn einer Knoten.

In einem Worst-Case Szenario könnten alle Knoten Nachbarn aller anderen Knoten sein. Aus diesem Grund besitzt es in einem Worst-Case Szenario die Laufzeitkomplexität

$$\mathcal{O}(N^2) \quad (15)$$

Bei der Findung einer möglichen Lösung werden zunächst einmal alle Knoten abhängig von ihrer Position sortiert ($\mathcal{O}(N \log N)$) und anschließend per Greedy Verfahren eine Route erstellt, wodurch sich die Laufzeitkomplexität im Durchschnitt sich zu folgender Komplexität bildet:

$$\mathcal{O}(N \log N + N \cdot \bar{L}) \quad (16)$$

Das liegt daran, dass die Abfrage nach einer Route immer in $\mathcal{O}(1)$ erfolgt.

In einem Worst-Case Szenario ($\bar{L} = N$) wäre es:

$$\mathcal{O}(N \log N + N^2) \quad (17)$$

Bei der eigentlichen Teil des ALNS-Verfahrens wird es komplexer (vgl. Hinweis in der Box). Aus diesem Grund wird hier von einem generellen ALNS-Verfahren, unabhängig von dem Entwickler/Hersteller, ausgegangen, wobei ebenfalls davon ausgegangen wird, dass alle ALNS-Modelle die gleiche Laufzeitkomplexität besitzen. Beim Nachfragen mit Gemini wurde folgende Aussage als wichtig betrachtet: „[D]ie exakten Komplexitäten [hängen] stark von den verwendeten Destruktions- [...] und Rekonstruktions-Operatoren [...] ab.“¹² In der Theorie macht diese Aussage Sinn, denn im Vergleich zu herkömmlichen Solvern (z.B. der ILP-Solver) werden keine Bit-Zustände oder andere Werte verwendet, um Graphen sowie Knoten darzustellen, sondern die Graphen selbst müssen von mir selbst erstellt oder zerstört werden. Alle Ergebnisse, ob ein Zustand akzeptiert oder abgelehnt wird, hängen daher nur von der daraus resultierenden Punktzahl/Strafe ab (**objective**). Aus diesem Grund wird daher zusätzlich noch folgende Annahme gemacht:

- Das Entscheidungsverfahren, ob ein Zustand akzeptiert oder abgelehnt wird, erfolgt in $\mathcal{O}(1)$
- Das Entscheidungsverfahren, ob der Graph zerstört oder repariert werden soll, erfolgt in $\mathcal{O}(1)$
- Alle Anpassungs- sowie Lernprozesse (v.a. im Zusammenhang mit der **SimulatedAnnealing** sowie **RouletteWheel**) erfolgen in $\mathcal{O}(1)$
- Die Überprüfung der Stop-Bedienung erfolgt in $\mathcal{O}(1)$

Gemini beschreibt daher die Laufzeitkomplexität des Programms als:¹³

$$\mathcal{O}(I \cdot (T + R + A)) \quad (18)$$

wobei I für die Anzahl der Iterationen, T für die Laufzeitkomplexität des Zerstörungsprozesses und R für die Laufzeitkomplexität des Reparaturprozesses steht. Der Faktor A steht hierbei für die Laufzeitkomplexität des Anpassungsprozesses, jedoch kann dieser Faktor aufgrund meiner Annahmen vernachlässigt werden. Die restlichen Sachen (alles außer Faktor A) machen weiterhin Sinn, da der Zerstörungsfunktion mind. einmal aufgerufen werden muss damit der Reparaturalgorithmus eine Auswirkung auf die Lösungsfindung hat. Daher resultiert es sich zu:

$$\mathcal{O}(I \cdot (T + R)) \quad (19)$$

¹²Vgl. Anhang1.pdf, Seite 1. Gemini Model 3 Flash

¹³Vgl. Anhang1.pdf, Seite 1. Gemini Model 3 Flash

Bei der Anzahl der Iterationen muss man hierbei bedenken, dass man diese selbst bestimmt. Durch die Verwendung der `MaxRuntime` als Stop-Bedingung kann man einen beliebigen Wert für `I` erwarten, dessen resultierende Dauer - in meinem Fall - ungefähr 0.2 Sekunden + Puffer beträgt.

Der Zerstörungsoperator (Faktor T) erstellt eine Kopie vom Zustand, wobei alle Knoten auf den neuen Zustand angepasst werden. Da bei der Kopie die Größe der Kopie von der Anzahl der Knoten im Graph abhängig ist, kann dieser Prozess mit dem Wert N beschrieben werden (V.a. da alle Knoten entweder in Routen dargestellt werden oder als „nicht zugewiesen“ gezeichnet werden müssen). Anschließend werden abhängig von `trees_to_remove` zufällige Routen ausgewählt, und aus den Routen werden anschließend zufällige Knoten entfernt. Da beim der Löschvorgang Knoten in eine Liste verschoben werden müssen (`routes.pop(treeId) → Faktor  $\bar{R}$`), hat die Anzahl der Knoten $|R|$ in den Routen ebenfalls einen Einfluss darauf. Für den Fall, dass die Route nach der Löschung keine Knoten mehr hat, hat die Größe der Route keinen Einfluss darauf, jedoch die Anzahl aller Routen N_R , da die Route aus der Liste aller Routen entfernt werden muss und alle weiteren Routen verschoben werden müssen. Aus diesem Grund bildet sich folgende durchschnittliche Laufzeitkomplexität:

$$\mathcal{O}\left(N + \text{trees_to_remove} \cdot \left(\bar{R} + \frac{N_R}{2}\right)\right) \quad (20)$$

Da alle Routen die gleiche Wahrscheinlichkeit haben, ausgewählt zu werden, ergibt sich der Faktor $\frac{N_R}{2}$. Da bei der Löschung vor allem Routen, die am Anfang der Liste stehen, höhere Auswirkungen auf die Laufzeit haben, während Routen, die am Ende der Liste stehen, kaum Auswirkungen auf die Laufzeit besitzen. Das liegt daran, dass die Positionen der Routen in der Liste bestimmen, wie viele Objekte während des Löschvorgangs verschoben werden müssen.

In einem Worst-Case Szenario wird davon ausgegangen, dass es jedes Mal die Knoten rausgelöscht werden, die am Anfang der Route stehen, und wenn Routen entfernt werden, es bei allen Fällen Routen sind, die am Anfang der Routenliste stehen. Aus diesem Grund bildet sich dadurch in einem Worst-Case Szenario folgende Laufzeitkomplexität:

$$\mathcal{O}(N + \text{trees_to_remove} \cdot (R + N_R)) \quad (21)$$

Beim Reparaturprozess (Faktor R) werden zunächst einmal alle nicht zugewiesenen Knoten zufällig vermischt und anschließend für jeden nicht eingefügten Knoten durchgegangen (Faktor N_{NzB}). Dabei werden ebenfalls die benachbarten Knoten zufällig vermischt, um anschließend die Routen aller benachbarten Knoten anzuschauen (Faktor $\bar{L}$) und zu versuchen, die Knoten an der ersten möglichen Stelle hinzuzufügen (Faktor $\bar{R}$), wobei bei jedem Versuch die Länge der Route erneut bestimmt wird. Aus diesem Grund bildet sich daher die durchschnittliche Laufzeitkomplexität

$$\mathcal{O}(N_{\text{NzB}} \cdot \bar{L} \cdot \bar{R}^2) \quad (22)$$

In einem Worst-Case Szenario müssen alle Knoten zugewiesen werden ($N_{\text{NzB}} = N$), alle Knoten sind Nachbarn voneinander und die maximale Routenlänge ist 0, wodurch durch alle benachbarten Knoten durchgegangen wird, um am Ende keine passende Route zu finden und daher eine neue Route zu erstellen ($\bar{L} = N$). Es bildet sich daher die Worst-Case Laufzeitkomplexität:

$$\mathcal{O}(N^2) \quad (23)$$

Somit wurden alle einzelnen Faktoren berücksichtigt. In dem durchschnittlichen Fall bildet sich daher die durchschnittliche Laufzeitkomplexität (bereits verkürzt):

$$\mathcal{O}\left(\frac{\Delta X \cdot \Delta Y}{\text{splitfactor}} + N \cdot \bar{d}^2 \cdot \bar{L} + N \log N + I \cdot \left(N + \text{trees_to_remove} \cdot \left(\bar{R} + \frac{N_R}{2}\right) + N_{\text{NzB}} \cdot \bar{L} \cdot \bar{R}^2\right)\right) \quad (24)$$

In einem Worst-Case Szenario würde es folgende Laufzeitkomplexität besitzen (bereits verkürzt):

$$\mathcal{O}\left(\frac{\Delta X \cdot \Delta Y}{\text{splitfactor}} + I \cdot (\text{trees_to_remove} \cdot (R + N_R) + N^2)\right) \quad (25)$$

4.2 Speicherplatzkomplexität

Zum Thema Speicherplatzkomplexität kann es in folgende Eigenschaften aufgeteilt werden:

- Die Erstellung der 2-dimensionalen Buckets
- Die Erstellung der Liste aus benachbarten Knoten
- Die Speicherung eines Zustands

Die Erstellung der 2-dimensionalen Buckets hängt hier wieder von der Differenz aus der größten zu den kleinsten x- und y-Koordinaten aller Knoten. Hinzu spielt der Wert von „splitfactor“ wieder eine Rolle, wodurch sich folgende Speicherplatzkomplexität ergibt:

$$\mathcal{O}\left(\frac{\Delta X \cdot \Delta Y}{\text{splitfactor}}\right) \quad (26)$$

Dabei gelten hier wieder: $\Delta X = x_{\max} - x_{\min}$ und $\Delta Y = y_{\max} - y_{\min}$. Bei der Erstellung der Liste aus benachbarten Knoten werden für jeden Knoten mind. 10 benachbarte Knoten in einer Liste gespeichert. Daher ergibt sich hier wieder eine durchschnittliche Speicherplatzkomplexität von:

$$\mathcal{O}(N \cdot \bar{L}) \quad (27)$$

In einem Worst-Case Szenario würde hier wieder bedeuten, dass alle Knoten Nachbarn zueinander sind. Aus diesem Grund bildet sich hier wieder die Worst-Case Speicherplatzkomplexität:

$$\mathcal{O}(N^2) \quad (28)$$

Bei der Speicherung der Zustände im Zusammenhang mit dem ALNS-Algorithmus kann man davon ausgehen, dass der ALNS-Algorithmus nur drei Zustände speichern kann:

- der Zustand mit der geringsten Strafe
- der neue Zustand
- der alte Zustand (für den Fall, dass ein neuer Zustand abgelehnt wird.)

Natürlich gibt es noch weitere Faktoren, v.a. im Zusammenhang mit dem Lern- und Anpassungsprozess. Jedoch kann man davon ausgehen, dass die alle eine Speicherplatzkomplexität von $\mathcal{O}(1)$ besitzen.¹⁴ Bei jedem Zustand werden höchstens N Elemente gespeichert, da alle Knoten in der Route aufgenommen werden und nur höchstens N Knoten gelöscht werden können. Aus diesem Grund hat der ALNS Algorithmus eine Speicherplatzkomplexität von $\mathcal{O}(N)$.¹⁵

Somit ergibt sich eine durchschnittliche Speicherplatzkomplexität von (bereits verkürzt):

$$\mathcal{O}\left(\frac{\Delta X \cdot \Delta Y}{\text{splitfactor}} + N \cdot \bar{L}\right) \quad (29)$$

Im Worst-Case-Fall beträgt die Speicherplatzkomplexität (bereits verkürzt):

$$\mathcal{O}\left(\frac{\Delta X \cdot \Delta Y}{\text{splitfactor}} + N^2\right) \quad (30)$$

4.3 Fragestellung Komplexität

Um die Verwendung eines Solvers begründen zu können, muss zunächst einmal begründet werden, weshalb das Problem in der Kategorie NP-Schwer eingestuft wird. Betrachtet man das Gießroboter Problem, so ist das Problem mindestens so schwer wie das „Capacitated Vehicle Routing Problem“ (kurz CVRP), welches die NP-Schwerheit „NP-Schwer“ durch das „Travelling Salesman Problem“ erbt.¹⁶ Das CVRP ist ein Optimierungsproblem und ein Teil des „Vehicle Routing Problem“ (kurz VRP) ist. Das VRP-Problem

¹⁴Vgl. Anhang1.pdf

¹⁵Vgl. Anhang1.pdf

¹⁶vgl. https://de.wikipedia.org/wiki/Vehicle_Routing_Problem

beschäftigt sich mit folgender Fragestellung: „What is the optimal set of routes for a fleet of vehicles to traverse in order to deliver to a given set of customers?“¹⁷ Dabei ist das Ziel des VRP-Problems, mithilfe eines Graphen $G = (V, E)$ - wobei die Knoten $V = \{v_0, v_1, \dots, v_n\}$ die Kunden mit n Anzahl der Kunden darstellen, der Knoten v_0 den Start- und Endpunkt einer Route darstellt und die Kanten $(i, j) \in E$ zwei Knoten mit der Gewichtsfunktion $w_E : E \rightarrow [0; \infty[$ verbinden - sowie einer bestimmten Anzahl an $K \in \mathbb{N}$ identische Fahrzeuge, die alle eine Route befahren dürfen, alle Kunden mit der minimalen Routenlänge zu versorgen.¹⁸ Bei der Erweiterung CVRP werden zusätzlich noch zu den Fahrzeugen $k \in K$ eine Kapazität $C_k \in [0; \infty[$ zugewiesen und alle Kunden i besitzen zusätzlich noch einen Bedarf $d_i \in [0; \infty[$, welcher von der Kapazität C_k der Fahrzeuge k abgedeckt werden muss.¹⁹

Vergleicht man das Gießroboter Problem mit der CVRP, so fallen einige Unterschiede der beiden Problemstellungen auf:

- Unterschiedliche Rolle des Depots v_0
- Unterschiedliche Beschränkung der Kapazität für die Fahrzeuge
- Unterschiedliche Funktionsweise des Bedarfs der Kunden
- Unterschiedliche Zielsetzungen der Funktion

Im Gießroboter Problem existiert kein Start- und Enddepot, an dem sich die Route endet, da man stattdessen nach einem Kreis aus Knoten sucht. Das CVRP-Problem besitzt ein Depot, das in allen Routen einmal beinhaltet sein muss und an einen Ort gebunden ist. Um die Ähnlichkeit mit dem CVRP-Problem zu beweisen, kann deshalb für jede Route mehrere fiktive Depots v_0 erstellt werden, die in dem vollständigen Graphen G für die zu betrachtende Route gültig ist und die Position des ersten Knotens $w \in V$ einer Route übernimmt, wodurch es ein Kantengewicht in der Höhe von: $w_E(v_0, w) = 0$ zu dem ersten Knoten besitzt. Hinzu wären die Depots weiterhin eindeutig an bestimmte Orte gebunden, deren Gültigkeit für eine Route gesehen wird, was den ähnlichen Effekt auf die Laufzeitkomplexität besitzt, wie die Bindung eines einzigen Depots an einen Ort.

In der Gießroboter Problem basieren sich die Beschränkungen auf die Längen der Routen, die nicht höher als C betragen darf, während bei der normalen CVRP die Fahrzeuge eine Kapazität C_k besitzen und die Fahrzeuge die einzelnen Kunden i einer Route einen bestimmten Bedarf d_i durch die Kapazität abdecken müssen. Jedoch lässt sich diesmal das Problem wieder anpassen, indem alle Fahrzeuge k die maximale Länge C als Kapazität $C_k = C$ zugewiesen bekommen und dabei alle Kunden dynamische Bedarfe zuweist, die den Abstand von dem Knoten der Kunden zu den Vorgängerknoten in der Route R darstellt. Dadurch kann die mathematische Berechnung der Kapazität aus:

$$\sum d_i \leq C_k; i \in R \quad (31)$$

zu folgende Formel umwandeln:

$$\sum w_{E_{i,i-1}} \leq C; i \in R \quad (32)$$

Aus der Formel ist zu erkennen, dass die hauptsächlichen Kostenfunktionen weiterhin vorhanden sind. Die Zielsetzung des standardmäßigen CVRP ist es, die Gesamtlänge der Route zu minimieren. In der Gießroboter Problem möchte man jedoch die Anzahl der Routen minimieren. Jedoch kann man hier begründen, dass die Minimierung der Gesamtlänge einer Route ein Nebenziel des Problems ist. Durch die Findung einer optimalen Route werden die Verbindungswege verkürzt und es passen mehr Knoten in eine Route rein, wodurch die durchschnittliche Anzahl der Knoten pro Route erhöht werden kann und dadurch auch potenziell weniger Routen benötigt wird, was eine positive Auswirkung auf die Hauptzielsetzung der Gießroboter Problem hat.

Beim standardmäßigen CVRP-Problem gibt es nur eine bestimmte Menge an Fahrzeugen K , während beim Gießroboter Problem so viele Fahrzeuge wie die Anzahl der Knoten im Graph G verwendet werden können. Jedoch lässt sich hier das Problem wieder umwandeln, indem man bei dem Gießroboter Problem nach der Frage fragt: „Ist es möglich, alle Kunden unter der Beachtung aller Einschränkungen mithilfe

¹⁷Quelle: https://en.wikipedia.org/wiki/Vehicle_routing_problem, geöffnet am 09.04.2026

¹⁸Vgl. https://de.wikipedia.org/wiki/Vehicle_Routing_Problem

¹⁹Vgl. https://de.wikipedia.org/wiki/Vehicle_Routing_Problem

von höchstens $|K|$ Fahrzeuge zu versorgen?“, wodurch die Suche nach einer unbekannten Fahrzeugmenge so umgewandelt werden kann, dass man über die Menge der Fahrzeuge weiß und so alle Eigenschaften aus der CVRP übernehmen kann.

Hinzu kommt noch, dass falls $|K| = 1$ ist, man nach einer Hamilton Kreis²⁰ suchen würde, weshalb das Problem schon bereits dadurch mindestens die Einstufung NP-Schwer hat.

Anhand der gesamten Begründung sollte bewiesen sein, dass das Gießroboter Problem mindestens so schwer wie das CVRP Problem ist und dadurch auch die Einstufung NP-Schwer übernimmt. D.h., dass nach aktuellem Stand²¹ keine Lösung in polynomieller Laufzeit existiert, wodurch heuristische Ansätze mithilfe von Solvern legitimiert werden.

Aufgrund der Fragestellung, die minimale Anzahl an Robotern zu verwenden, kann bei der Überprüfung einer Antwort folgende Frage gestellt werden: „Ist die aktuelle Lösung tatsächlich die minimale mögliche Anzahl an Robotern, die man verwenden muss?“. Diese Frage lässt sich jedoch nicht in einer polynomiellen Zeit beantworten, da es keine Möglichkeit gibt, auf solche Eigenschaften zu überprüfen. Die einzige Möglichkeit, solche Fragen zu beantworten, ist entweder durch:

- Die Überprüfung aller Möglichkeiten in einer exponentiellen Zeit
- Die Erstellung einer neuen Lösung, die weniger Routen verwendet. (Jedoch kann hier nur bewiesen werden, dass eine Lösung nicht die beste Lösung ist. Falls keine bessere Lösung generiert werden kann, bleibt diese Frage unbeantwortet.)

Aus diesem Grund lässt sich das Problem in der Kategorie: NP-Schwer zuordnen.

5 Werkzeuge

Für die Umsetzung wurden folgende Werkzeuge verwendet:

- GeoGebra ([geogebra.org/calculator](https://www.geogebra.org/calculator))
- ALNS (pypi.org/project/alns/ github.com/N-Wouda/ALNS)
- ALNS Dokumentation (<https://alns.readthedocs.io>)
- matplotlib (pypi.org/project/matplotlib/)
- StackOverflow
- Gemini (gemini.google.com/)

Es wurde GeoGebra verwendet, um mathematische Gleichungen zu erstellen und im Zusammenhang mit der Knotenmenge zu analysieren. Dies wurde später angewendet, um einen flexiblen Zerstörungsgrad in Abhängigkeit der Knotenzahl zu erstellen.

Es wurde alns als Solver für die „Adaptive Large Neighborhood Search“ Probleme verwendet. Dabei handelt es sich um einen Solver, der die Generierung einer guten Lösung möglich macht. Während der Implementierung wurden Ausschnitte aus der öffentlichen Dokumentation übernommen, gelernt, experimentiert und anschließend für das Problem angepasst. Die Quellen der Codes sind alle entweder im Quellcode und/oder in der Umsetzung genannt worden und stammen aus folgender Website: alns.readthedocs.io.

Es wurde matplotlib verwendet, um die Lösungen sowie die Beispiele zu visualisieren.

Es wurde StackOverflow verwendet, um Implementierungsprobleme - vor allem im Zusammenhang mit Numpy - zu lösen. Dadurch war es möglich, die Buckets Lösung auf einen 2-dimensionalen Raum erfolgreich umzusetzen. Alle Lösungen aus StackOverflow wurden dabei per Debugging auf ihre Funktionalität überprüft.

Es wurde Gemini verwendet, um eine Laufzeitanalyse für das ALNS Modul durchzuführen. Das ALNS Modul hat seine Laufzeit- und Speicherkapazität nicht öffentlich angegeben, weshalb die Untersuchung mit der KI erfolgt ist. Alle Aussagen wurden ausführlich von mir überprüft. Ein Nutzungsverlauf ist in der Datei „Anlage1.pdf“ zu finden.

²⁰Vgl. <https://de.wikipedia.org/wiki/Hamiltonkreisproblem>

²¹Solange das NP versus P Problem noch unbeantwortet bleibt. Vgl. https://en.wikipedia.org/wiki/P_versus_NP_problem

6 Beispiele

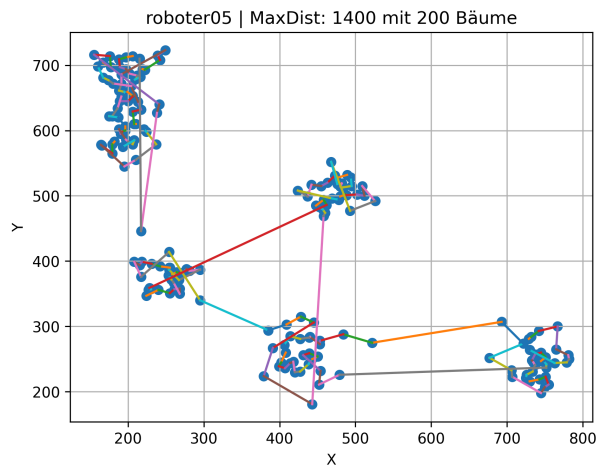
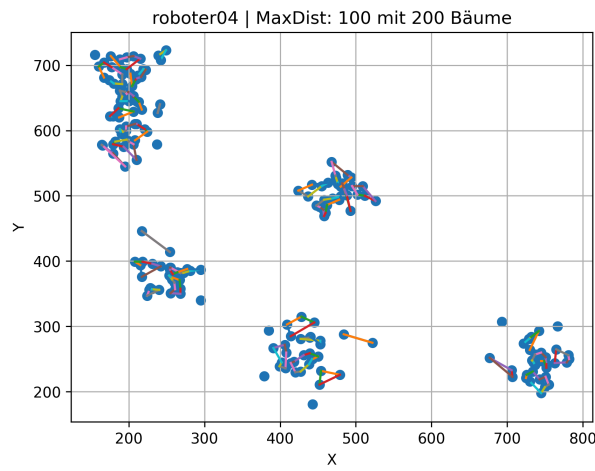
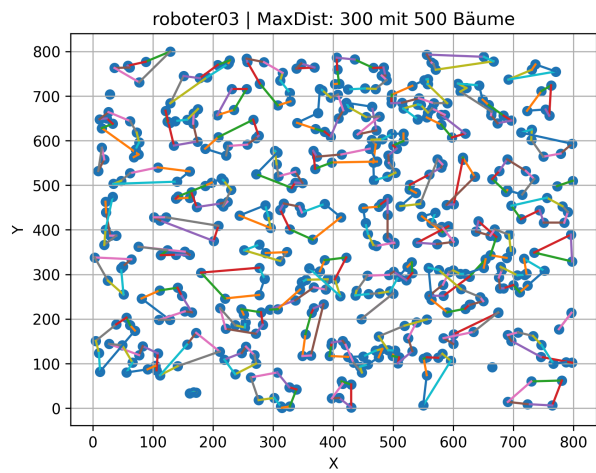
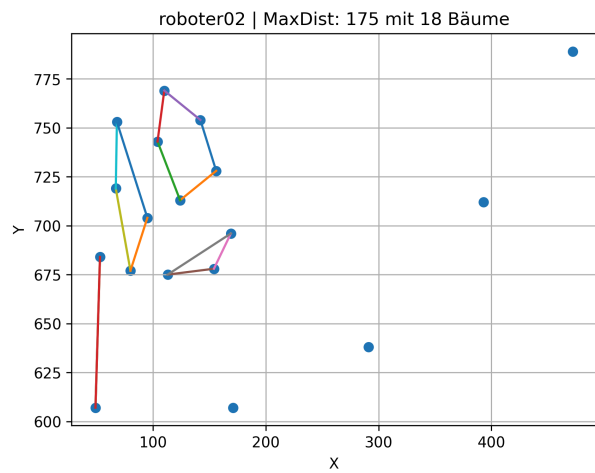
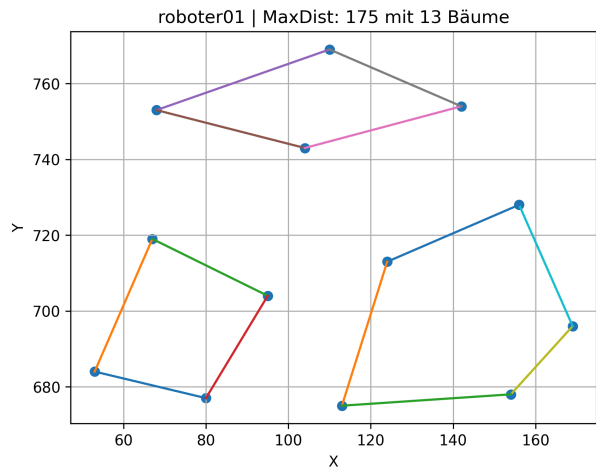
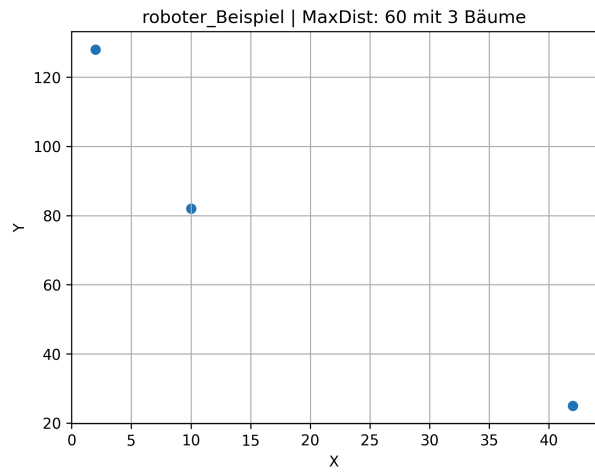
Für die Beispiele wurden noch einige zusätzliche Beispiele erstellt, um Grenzszenarien darzustellen und um die Robustheit der Lösung zu beweisen. Alle Eingaben sowie Ausgaben sind im Ordner zu finden.

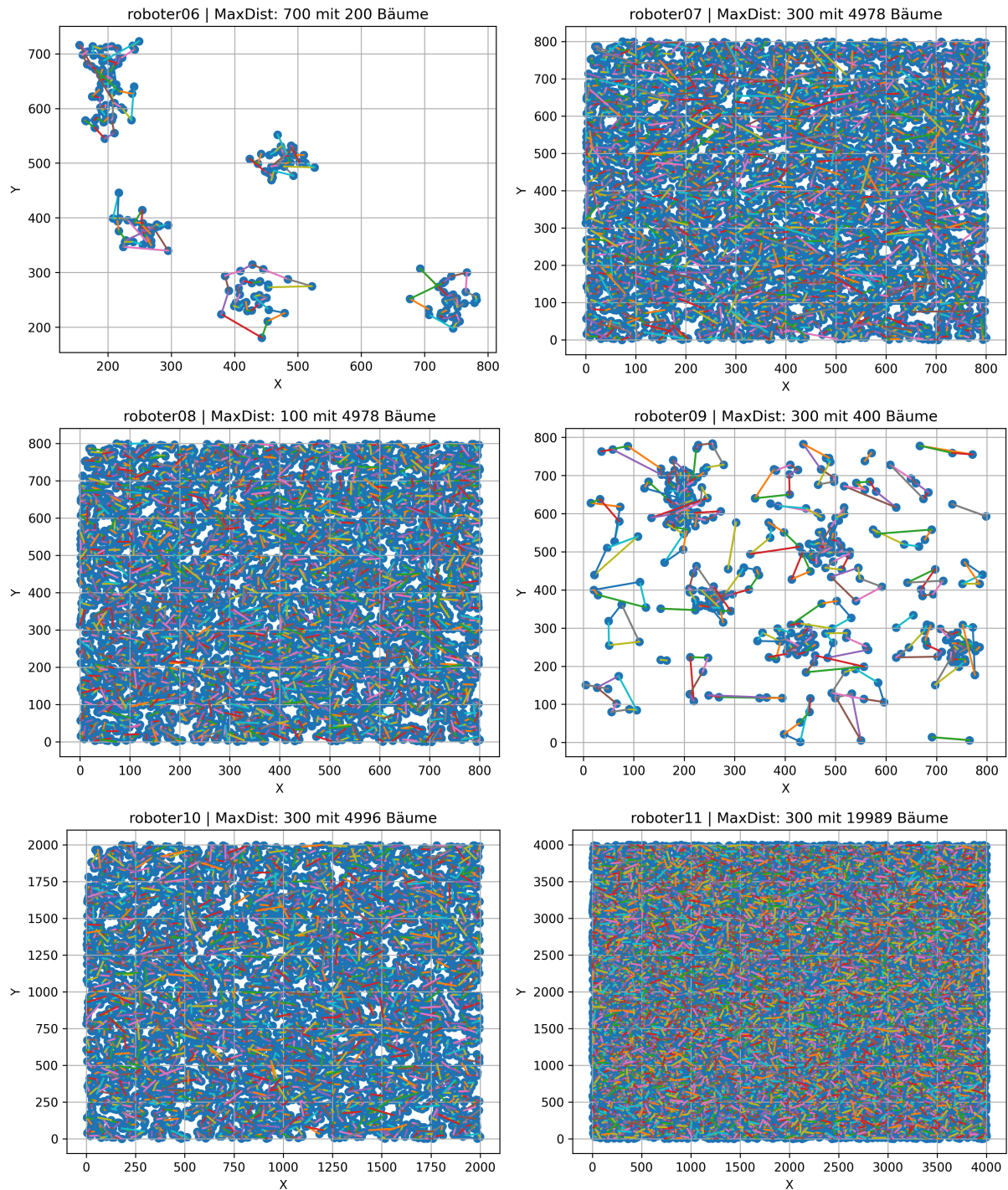
6.1 Vorgegebene Beispiele

Aufgabe	Ausgabe	Kommentar
roboter_Beispiel.txt	3 60 0 10 82 1 0 42 25 2 0 2 128 3	Execution time: 0.201454s
roboter01.txt	3 175 128 80 677 11 13 4 3 157 110 769 5 1 2 12 173 154 678 7 8 9 6 10	Execution time: 0.202289s
roboter02.txt	8 175 162 142 754 12 9 6 2 5 124 113 675 10 7 8 164 80 677 11 4 1 3 154 53 684 13 14 0 393 712 17 0 472 789 18 0 171 607 15 0 291 638 16	Execution time: 0.202289s

Aufgabe	Ausgabe	Kommentar
roboter03.txt	67	Execution time: 0.287054s
roboter04.txt	39	Execution time: 0.223850s
roboter05.txt	<pre> 4 1400 1388 215 710 1 26 22 31 25 6 33 28 18 13 10 3 8 7 21 32 9 12 29 30 5 2 179 175 188 16 11 173 174 189 177 187 169 171 199 178 196 190 176 184 191 182 170 193 181 180 183 197 185 168 186 172 192 200 198 194 23 19 27 24 15 20 17 34 14 4 195 79 1318 752 237 35 37 64 45 48 57 66 61 44 47 60 36 59 50 65 54 56 63 43 67 62 41 51 55 68 39 58 53 38 40 49 42 52 46 109 120 102 125 110 106 126 118 116 128 105 127 130 124 114 117 132 107 113 122 112 108 131 111 103 134 1380 228 359 69 88 96 93 70 82 81 71 99 90 84 87 76 98 92 94 97 95 74 100 78 73 86 83 101 85 91 80 75 72 77 89 123 133 119 115 104 121 129 159 158 156 167 141 144 135 428 476 518 136 147 161 155 142 163 160 148 145 140 146 162 150 138 152 153 143 164 154 166 139 149 137 151 165 157 </pre>	Execution time: 0.248018s Durch die großen Bucket-Räume können weiterhin entfernte Verbindungen entstehen, was in diesem Fall eine positive Auswirkung hat.
roboter06.txt	<pre> 6 700 697 179 579 170 180 181 193 183 197 176 184 192 200 194 13 23 18 28 1 26 22 31 34 17 25 6 33 10 3 8 7 21 32 9 12 29 11 5 30 2 179 175 188 16 173 189 174 177 178 187 169 171 199 196 190 191 182 372 167 704 20 14 172 186 168 185 4 195 198 19 27 24 15 597 763 244 42 35 64 45 52 46 51 66 57 37 48 61 44 47 60 36 59 50 65 54 56 63 43 67 62 41 55 68 39 58 53 38 40 49 674 255 390 83 101 77 76 71 81 85 91 75 79 80 72 88 69 97 89 93 96 70 82 99 90 84 87 98 92 94 95 74 100 78 73 86 </pre>	Execution time: 0.225322s

Aufgabe	Ausgabe	Kommentar
	687 409 303 133 119 115 120 109 125 102 110 106 126 118 116 128 105 127 130 124 114 117 132 107 113 122 112 108 131 111 134 103 129 121 104 123 575 493 477 165 143 151 149 137 163 139 148 160 144 153 161 157 166 164 154 142 155 136 147 145 140 146 162 150 138 152 141 167 156 158 159 135	
roboter07.txt	220	Execution time: 0.995591s Durch die Abhängigkeit des „splitfactor“ mit der Dichte, können weiterhin effektiv Lösungen gebildet werden, die viele Knoten in eine Fläche beinhalten.
roboter08.txt	660	Execution time: 0.893655s
roboter09.txt	52	Execution time: 0.292844s Durch zufälliges Entfernen einer Knoten kann man hier deutlich erkennen, dass solche Maßnahmen langfristig zu besseren Ergebnissen führen.
roboter10.txt	562	Execution time: 0.694615s
roboter11.txt	2248	Execution time: 2.343852s Durch das Buckes-System können auch sehr viele Knoten auf dem Feld schnell berechnet werden.

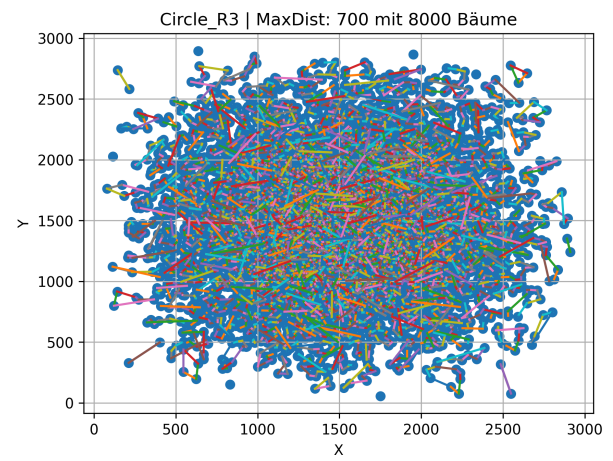
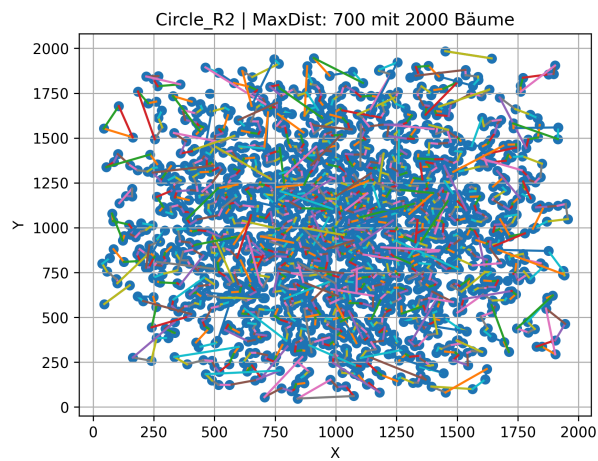
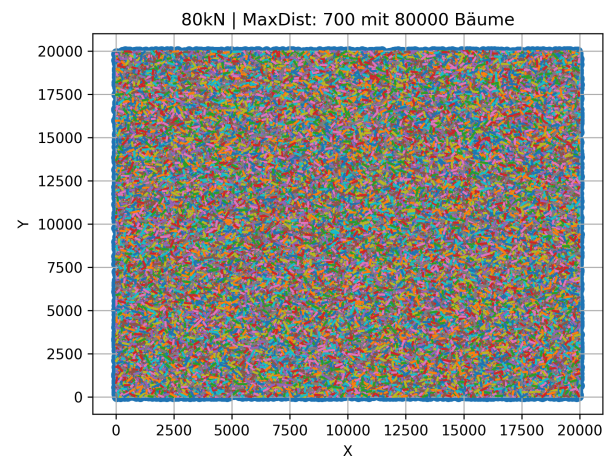
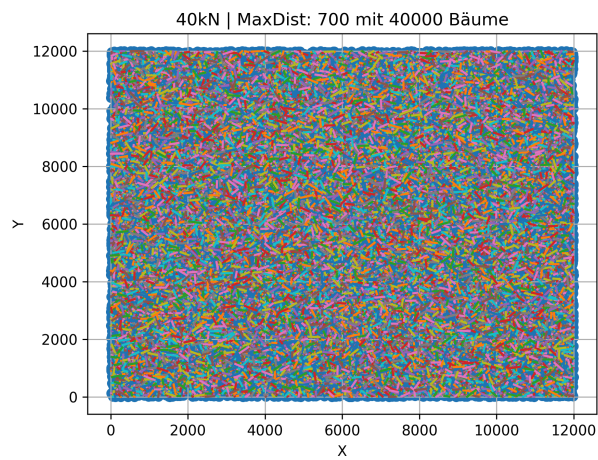
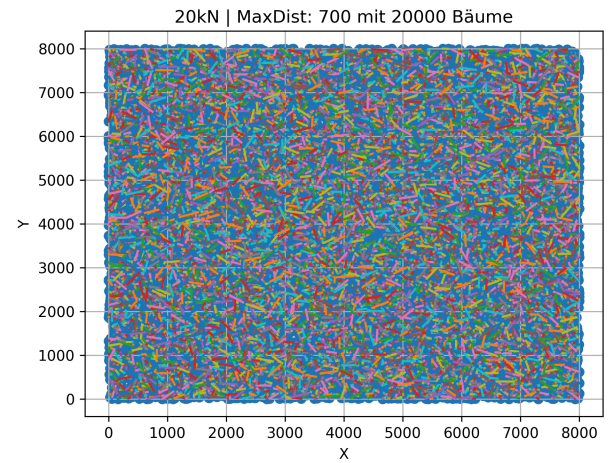
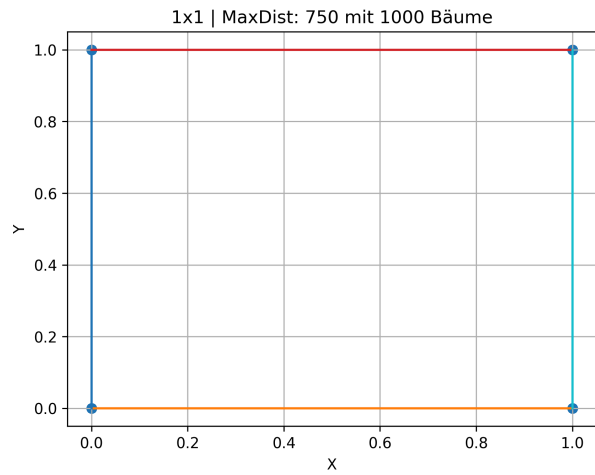


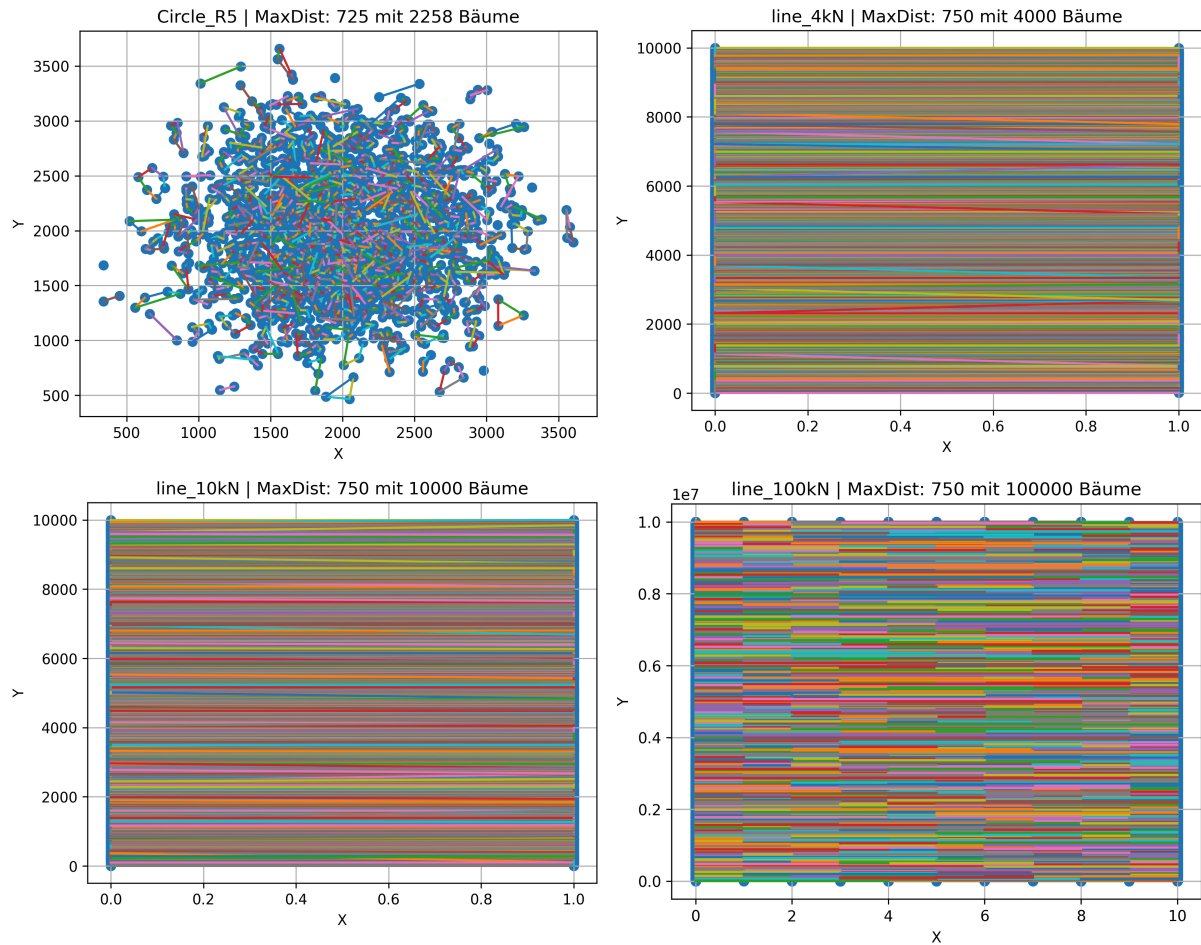


6.2 Eigene Beispiele

Aufgabe	Ausgabe	Kommentar
1x1.txt	1	Execution time: 1.099169s In diesem Beispiel wird ein Worst-Case Szenario simuliert mit 1000 Bäumen, wo alle Knoten zu allen anderen Knoten Nachbarn sind, da das Feld 2x2 groß ist.

Aufgabe	Ausgabe	Kommentar
20kN.txt	1937	Execution time: 2.098516s Beispiel mit 20000 Bäumen, verteilt auf einer großen Fläche. MaxDist ist 700
40kN.txt	4129	Execution time: 4.659987s Beispiel mit 40000 Bäumen, verteilt auf eine größeren Fläche. MaxDist ist 700
80kN.txt	9725	Execution time: 6.775176s Beispiel mit 80000 Bäumen, verteilt auf eine noch größere Fläche. MaxDist ist 700 Dies soll zeigen, dass das Programm für eine große Anzahl an Bäume problemlos funktioniert, und dass die nahliegenden Bäume Filter für so etwas geeignet ist.
Circle_R2.txt	133	Execution time: 0.750877s Beispiel mit 2000 Bäumen, verteilt auf eine kleine Fläche wobei in der Mitte mehr Bäume sind. MaxDist ist 700
Circle_R3.txt	336	Execution time: 5.529022s Beispiel mit 8000 Bäumen, verteilt auf eine kleine Fläche wobei die meisten Bäume in der Mitte sind. MaxDist ist 700
Circle_R5.txt	188	Execution time: 0.700649s Beispiel mit 2258 Bäumen, verteilt auf eine mittlere Fläche wobei sehr viele Bäume in der Mitte sind. MaxDist ist 725 Dadurch soll gezeigt werden, dass das Programm bei ungewöhnlicher Dichteverteilung ebenfalls gute Ergebnisse ausgeben kann.
line_4kN.txt	46	Execution time: 0.795282s 4000 Bäume auf einer 10001x2 Fläche. MaxDist ist 750
line_10kN.txt	73	Execution time: 2.621896s 10000 Bäume auf einer 10001x2 Fläche. MaxDist ist 750
line_100kN.txt	23515	Execution time: 8.460881s 100000 Bäume, auf einer 10000000x10 Fläche. MaxDist ist 750





7 Quellcode

Aus Platzgründen wurden redundante Textstellen stark verkürzt (gekennzeichnet durch [...]) oder weggelassen. Die wichtigsten Strukturen sind weiterhin vorhanden. Für den vollständigen Quellcode, sollte die im Abgabe-Ordner findbare Quelltext Datei angeschaut werden.

```
def solve(filename):
    # Daten einlesen
    # -> positions, nTrees, maxDist
    with open(f"A2_Gießroboter\\{filename}.txt", "r") as f:
        Input = f.read()
        lines = Input.strip().split("\n")
        maxDist = float(lines[0])
        nTrees = int(lines[1])
        positions = []
        pos_to_ids = []

        minx = 1e9
        maxx = 0
        miny = 1e9
        maxy = 0

    for line in lines[2:]:
        xy = line.split(" ")
        pos_to_ids.append(xy[0])

        minx = min(minx, int(xy[1]))
        maxx = max(maxx, int(xy[1]))
        miny = min(miny, int(xy[2]))
```

```

maxy = max(maxy, int(xy[2]))

positions.append((float(xy[1]), float(xy[2])))
del Input, lines, line, xy

# 2D-Bucket erstellen
# Abstand abhängig von der Dichte berechnen
density = nTrees / ((maxx - minx) * (maxy - miny)) * 100
splitfactor = 100
if density > 0.1:
    splitfactor = 49
if density > 0.5:
    splitfactor = 23

# 2D-Bucket generieren
grid = np.frompyfunc(list, 0, 1)(np.empty((math.ceil((maxx - minx)/splitfactor) + 1,
math.ceil((maxy - miny)/splitfactor) + 1), object))

# Alle Bäume in die Buckets einspeichern und für alle Bäume ihre Bucket-Koordinate
speichern
idtogrid = []
for i in range(nTrees):
    x, y = positions[i]
    gridx = math.floor((x - minx) / splitfactor)
    gridy = math.floor((y - miny) / splitfactor)
    grid[gridx, gridy].append(i)
    idtogrid.append((gridx, gridy))

# Hilfsfunktion: Alle Bäume aus der Nähe eines Baumes zurückgeben
def getNearbyTrees(id, d):
    # id: Baumid
    # d: Maximale Buckets Distanz zu Bucket des Baums
    x, y = idtogrid[id]
    x1 = max(0, x - d)
    x2 = min(grid.shape[0], x + d + 1)
    y1 = max(0, y - d)
    y2 = min(grid.shape[1], y + d + 1)

    # Alle Bäume aus naheliegenden Buckets zurückgeben.
    return [a for b in grid[x1:x2, y1:y2].ravel() for a in b]

# Für alle Bäume naheliegende Bäume einspeichern.
nearbyList = []
for i in range(nTrees):
    d = 2
    while True:
        # Für jeden Baum muss mindestens 10 oder alle naheliegenden Bäume eine Referenz
        gespeichert werden.
        nearby = getNearbyTrees(i, d)
        if len(nearby) < min(10, nTrees):
            d += 1
            continue
        nearby.remove(i)
        nearbyList.append(nearby)
        break

# Hilfsfunktion: Distanz zwischen Knoten u und v speichern, wobei nur nötige Distanzen
(abhängig von naheliegenden Knoten) gespeichert werden.
storage = [{i} for i in range(nTrees)]
def dist(a, b):
    # Überprüfen ob die Distanz schon bereits berechnet wurde.
    value = storage[a].get(b, None)

```

```

    if value is None:
        # Falls es noch nicht berechnet wurde: Distanz berechnen und für zukünftige
        # Verwendung speichern.
        value = calc_dist(positions[a], positions[b])
        storage[a][b] = value
        storage[b][a] = value
    return value

# Initialisierung: Eine mögliche Lösung berechnen
pre_positions = []
for i in range(nTrees):
    x, y = positions[i]
    pre_positions.append((x, y, i))

# Hilfsfunktion für die Sortierung
def sortxy(a, b):
    return (a[0] + a[1]) - (b[0] + b[1])
pre_positions = sorted(pre_positions, key=cmp_to_key(sortxy), reverse=True)

# Überprüfen, ob ein Knoten schon bereits eine Route bekommen hat.
pre_calculated = [0] * nTrees

# Anfängliche Route sowie Zuweisungen initialisieren.
init_routes = []
init_assignment = [None] * nTrees

# Für alle Bäume mithilfe eines Greedy Algorithmus eine Route zuweisen.
for i in range(nTrees):
    if pre_calculated[i]: continue
    pre_calculated[i] = 1

    pre_route = []
    pre_route.append(i)

    pre_length = 0
    # Failsafe: Falls es nur einen Baum existiert.
    while True:
        # Beste nächste Knoten berechnen.
        pre_bestDist = 1e9
        pre_bestId = -1
        for j in nearbyList[pre_route[-1]]:
            # Durch alle Nachbarn der zuletzt eingefügten Knoten durchgehen und die
            # nächstgelegenen noch nicht eingefügten Knoten merken.
            if pre_calculated[j]: continue
            pre_Dist = dist(j, pre_route[-1])
            if pre_Dist < pre_bestDist:
                pre_bestDist = pre_Dist
                pre_bestId = j
        # Fall: Es gibt keine unbesuchten Nachbarn
        if (pre_bestId == -1): break
        # Überprüfen, ob der Zeitlimit durch das Einfügen der besten Knoten weiterhin
        # gehalten wird.
        if (pre_length + pre_bestDist + dist(pre_bestId, pre_route[0]) <= maxDist):
            pre_length += pre_bestDist
            pre_calculated[pre_bestId] = 1
            pre_route.append(pre_bestId)
        else:
            break

    # Route speichern und alle Bäume auf ihre Route zuweisen.
    init_routes.append(pre_route)
    for j in pre_route:
        init_assignment[j] = init_routes[-1]

```

```

# Adaptive Large Neighbor Search initialisieren
# Quelle: https://alns.readthedocs.io/en/latest/examples/capacitated_vehicle_routing_
problem.html#Solution-state
class VRP(alns.State):
    # Diese Klasse stellt einen möglichen Zustand der Routenaufteilung dar.
    def __init__(self, routes, unassigned = None, assignment = None):
        # Neuen Zustand erstellen und von anderen Zuständen trennen.
        self.routes = routes
        if unassigned is None:
            self.unassigned = []
        else:
            self.unassigned = unassigned
        if assignment is None:
            self.assignment = []
        else:
            self.assignment = assignment

    # Zielfunktion: Ziel -> minimieren
    def objective(self):
        nRoutes = sum(1 for i in self.routes if len(i) > 0)
        return len(self.unassigned) * maxDist * 1.5 + nRoutes * maxDist * 1.3 +
sum(self.route_cost(route) for route in self.routes)

    @property
    def cost(self):
        return self.objective()

    # Länge einer Route berechnen
    @staticmethod
    def route_cost(route):
        cost = dist(route[0], route[-1])
        for i in range(len(route) - 1):
            cost += dist(route[i], route[i + 1])
        return cost

    # Maximale Zerstörungsgrad abhängig von der Anzahl der Bäume berechnen
    # Quelle: https://alns.readthedocs.io/en/latest/examples/capacitated_vehicle_routing_
    problem.html#Destroy-operators
    if nTrees <= 50:
        math_destruction = 0.2
    else:
        math_destruction = 0.2 - 0.0011 * (nTrees - 50)
    degree_of_destruction = max(0.05, math_destruction)

    # Maximale Anzahl der zerstörten Bäume abhängig von der Anzahl der Bäume berechnen
    trees_to_remove = int((nTrees) * degree_of_destruction)

    # Zerstörungsfunktion: Mehrere zufällige Knoten aus ihrer Route rauswerfen.
    def multi_random_removal(state, rng):
        destroyed_Routes = [route.copy() for route in state.routes if len(route) > 0]
        assignment = [None] * nTrees
        for route in destroyed_Routes:
            for tree in route:
                assignment[tree] = route
        destroyed = VRP(destroyed_Routes, state.unassigned.copy(), assignment)

        for i in range(rng.randint(1, int(trees_to_remove) + 2)):
            if not destroyed.routes:
                break
            routeId = rng.randint(len(destroyed.routes))
            treeId = rng.randint(len(destroyed.routes[routeId]))

```

```

        destroyed.assignment[destroyed.routes[routeId][treeId]] = None
        destroyed.unassigned.append(destroyed.routes[routeId].pop(treeId))
        if len(destroyed.routes[routeId]) == 0:
            destroyed.routes.pop(routeId)
    return destroyed

# Nicht zugewiesene/rausgeworfene Knoten einer Route zuweisen.
def fast_repair(state, rng):
    rng.shuffle(state.unassigned)
    while len(state.unassigned) != 0:
        tree = state.unassigned.pop()
        inserted = False

        rng.shuffle(nearbyList[tree])
        for nearbyTree in nearbyList[tree]:
            route = state.assignment[nearbyTree]
            if route is None:
                continue
            for i in range(len(route) + 1):
                newRoute = route[:i] + [tree] + route[i:]
                if VRP.route_cost(newRoute) <= maxDist:
                    route.insert(i, tree)
                    state.assignment[tree] = route
                    inserted = True
                    break
            if inserted: break
        else:
            # Falls keine Route gefunden werden konnte: Neue Route erstellen.
            state.routes.append([tree])
            state.assignment[tree] = state.routes[-1]
    return state

# Anfangszustand mit der Anfangslösung erstellen
state = VRP(init_Routes, assignment=init_assignment)

# Einzelne Operatoren anwenden und speichern
alns_obj = alns.ALNS(rnd.RandomState(42))
alns_obj.add_destroy_operator(multi_random_removal)
alns_obj.add_repair_operator(fast_repair)

select = alns.select.RouletteWheel([3, 1, 1, 0.1], 0.80, 1, 1)
accept = alns.accept.SimulatedAnnealing(0.8, 0.01, 0.85)
stop = alns.stop.MaxRuntime(0.2)

# Lösung berechnen
result = alns_obj.iterate(state, select, accept, stop)
best = result.best_state

routes = [route for route in best.routes if route]
print(f"Solution: {len(routes)} routes")

# Failsafe Check: Alle Bäume sollen eine Route zugewiesen bekommen haben.
counter = 0
for route in routes:
    counter += len(route)
assert counter == nTrees

# Ergebnis in der Datei speichern.
with open("Ausgabe\\" + filename + "_out.txt", "w") as f:
    f.write(f"{len(routes)}\n")
    f.write(f"{int(maxDist)}\n")

```

```
for i in range(len(routes)):
    f.write(f"{int(VRP.route_cost(routes[i]))}\n") #DEBUG
    #f.write(f"{int(Lengths[i]**0.5)}\n") # floor
    posX, posY = positions[routes[i][0]]
    f.write(f"{int(posx)} {int(posy)}\n")
    for j in routes[i]:
        f.write(f"{pos_to_ids[j]} ")
    f.write("\n")
```